QUIZ RUNNER OVER A RESTRICTED NETWORK



Session (2021-2025)

Bachelor of Studies in Artificial Intelligence

Submitted by

Khizer Dawood
303-211076

Muhammad Uzair
303-211013

Ramish Khan
303-211031

Supervised By

Mr. Miraj Gul
Lecturer, Department of CS & IT

Hazara University Mansehra

CamScanner

Approval Certificate

This is to certify that we have reviewed the project report titled **"Quiz Runner Over a Restricted Network"** submitted by the following students of BSAI, Fall 2025, Hazara University, Mansehra.

Name	Roll No.
Khizer Dawood	303-211076
Muhammad Uzair	303-211013
Ramish Khan	303-211031

It is our judgment that this project report meets the required standards and is hereby approved for acceptance by the Department of CS & IT, Hazara University, Mansehra.

COMMITTEE

1. **Supervisor**

Miraj Gul
Lecturer Deptt of IT
Hazara University

2. **Chairman**

3. **External Examiner**

Declaration

It is hereby declared that this project, neither as a whole nor as a part has been copied out from any source. It is further declared that we developed this software and this report entirely based on our personal efforts made under the sincere guidance of our project Supervisor. If any part of this software is proved to be copied or found to be a report of some other, we are standby the consequences. No portion of the work presented in this report has been submitted in support of any application from any other degree of qualification of any other university or institute of learning. We further declare that this software and all associated documents, reports and records are submitted as partial requirement for the degree of BSAI (Bachelor of Studies in Artificial Intelligence). We therefore understand and transfer copyrights for these materials to Hazara University Mansehra.

CamScanner

Acknowledgements

We would like to express our deepest gratitude to our teachers and supervisors for providing us the invaluable opportunity to work on the project **"Quiz Runner Over a Restricted Network."** This endeavor not only enhanced our technical skills but also broadened our understanding of network-based applications. Our sincere thanks go to **Hazara University Mansehra** for fostering an environment of innovation and learning through its well-structured curriculum. We are also immensely grateful to the faculty of the **CS&IT Department** for their unwavering support and guidance, without which this project would not have been possible. A special note of appreciation goes to our project supervisor, **Mr. Miraj Gul**, for his constant mentorship, insightful feedback, and patience throughout the development process. His expertise and encouragement were instrumental in the successful completion of this project.

CamScanner

Dedication

We dedicate this work, first and foremost, to our loving parents our pillars of strength whose unwavering prayers, boundless affection, and endless sacrifices have guided us to this milestone. Their belief in us, even in moments of doubt, has been the light that steered us forward.

To our teachers and mentors, we owe the gift of knowledge and wisdom. You armed us with "real eyes" the vision to discern, the courage to question, and the resolve to persevere in a world that often tests the spirit.

We also extend this dedication to our friends and well-wishers, whose encouragement fueled our journey, and to Almighty Allah, the ultimate source of wisdom and grace.

May this achievement honor those who shaped us and inspire those who follow.

CamScanner

Project in Brief

Project Title	: QUIZ RUNNER OVER A RESTRICTED NETWORK
Developed By	: Khizer Dawood, Muhammad Uzair, Ramish Khan
Supervised By	: Mr. Miraj Gul
Tools Used	: Visual Studio Code, Streamlit, Python
System Used	: Lenovo core i5, 12th (Gen), 8 GB RAM
Operating System	: Microsoft Windows 10
Starting Date	: October 2024
Completion Date	: May 2025

CamScanner

Abbreviations

MCQs	Multiple Choice Questions
PDF	Portable Document Format
API	Application Programming Interface
SDK	Software Development Kit
CRUD	Create, Read, Update, Delete
UID	Unique Identifier
DB	Database
UI	User Interface

CamScanner

TABLE OF CONTENTS

Chapter 1

Introduction 1

1.1 Introduction: 2

1.2 Security: 2

1.2.1 User Authentication and Access Control: 2

1.2.2 Encrypted Data Handling: 3

1.2.3 Firestore Security Rules: 3

1.2.4 Restricted Network Compatibility: 3

1.3 Background of the Project: 3

1.4 Scope of the Project: 4

1.5 Feasibility of the Project: 5

1.5.1 Economic Feasibility: 5

1.5.2 Technical Feasibility: 5

1.6 Introduction of the Tools Used in the Project: 6

1.6.1 Streamlit: 6

1.6.2 Firebase (Firestore & Authentication): 6

1.6.3 Visual Studio Code (VS Code): 7

1.6.4 OpenAI GPT-4 API: 7

1.6.5 PyMuPDF (fitz): 8

1.6.6 Python: 8

1.6.7 Requests Library: 8

1.6.8 Bcrypt: 9

1.6.9 Python-dotenv: 9

CamScanner

1.7 Report of Introduction: 9

Chapter 02

Requirements Specification 10

2.1 Introduction: 11

2.2 Existing System: 11

2.3 Limitations of the Existing System: 11

2.3.1 High Dependence on Internet Connectivity: 11

2.3.2 Lack of Automation: 11

2.3.3 Security Vulnerabilities: 12

2.3.4 Inability to Generate Smart MCQs: 12

2.3.5 Lack of Quiz Timing and Duration Control: 12

2.4 Proposed System: 12

2.5 Functional Requirements: 12

2.5.1 User Authentication and Account Management: 13

2.5.2 Teacher Functionalities: 13

2.5.3 Student Functionalities: 14

2.5.4 Administrative Functionalities: 15

2.5.5 Use case Diagram: 16

2.5.6 Data Flow Diagram: 17

2.6 Non-Functional Requirements: 17

2.6.1 Security: 17

2.6.2 Availability: 18

2.6.3 Performance: 18

2.6.4 Scalability: 18

CamScanner

2.6.5 Maintainability:..... 19
2.6.6 Usability:..... 19
2.6.7 Compatibility:..... 19
Chapter 3
Design of the Proposed System..... 20
3.1 System Architecture:..... 21
3.1.1 Components:..... 21
3.1.2 Architecture Diagram..... 23
3.2 High-Level Design:..... 23
3.2.1 Presentation Layer:..... 23
3.2.2 Application Logic Layer:..... 24
3.2.3 Data Layer:..... 24
3.2.4 Key Features of Logical Architecture..... 24
3.2.5 Logical Architecture Diagram..... 25
3.3 Low Level Design:..... 25
3.3.1 Class Diagram..... 26
3.3.2 Class Descriptions:..... 26
3.4 Optimized Database Design:..... 27
Chapter 04
Testing..... 28
4.1 Test Case Scenarios:..... 29
4.1.1 Authentication Module:..... 29
4.1.2 MCQs Generation Module:..... 34
4.1.3 PDF Text Extraction Module:..... 35
4.1.4 User Authentication Module:..... 36

CamScanner

Chapter 05

Results of Screenshots..... 36

5.1 Login Screen:..... 37

5.2 Signup Screen:..... 38

5.3 Reset Password: 39

5.4 Teacher Dashboard 40

5.5 Create And Publish Quiz:..... 41

5.6 Register Students: 42

5.7 View Student Scores: 43

5.8 Student Dashboard:..... 44

5.9 Past Quizes Record:..... 45

Chapter 6

References and Manual Training..... 46

6.1 References: 47

6.2 User Manual:..... 48

CamScanner

LIST OF FIGURES

Figure 2.1 Use Case Diagram 16

Figure 2.2 : Data Flow Diagram 17

Figure 3.1 : Architecture Diagram 23

Figure 3.2 : Logical Architecture Diagram 25

Figure 3.3 : Class Diagram 26

Figure 3.4 : Optimized Database Design 27

Figure 5.1 : Login Screen 37

Figure 5.2 : Signup Screen 38

Figure 5.3 : Reset Password 39

Figure 5.4 : Teacher Dashboard 40

Figure 5.5 : Create and Publish Quiz 41

Figure 5.6 : Register Students 42

Figure 5.7 : View Student Scores 43

Figure 5.8 : Student Dashboard 44

Figure 5.9 : Past Quizes Record 45

CamScanner

LIST OF TABLES

TABLE 1: AUTHENTICATION MODULE: 29

TABLE 2 : USER SIGNUP - EMAIL ALREADY EXISTS: 30

TABLE 3 : USER LOGIN – SUCCESSFUL: 30

TABLE 4 : USER LOGIN – INCORRECT PASSWORD: 31

TABLE 5 : PASSWORD RESET - VALID EMAIL: 32

TABLE 6 : PASSWORD RESET - INVALID EMAIL: 32

TABLE 7 : FETCH USER INFO - USER EXISTS: 33

TABLE 8 : FETCH USER INFO - USER DOES NOT EXIST: 34

TABLE 9 : GENERATE MCQS FROM INPUT TEXT: 35

TABLE 10 : EXTRACT TEXT FROM UPLOADED PDF FILE: 36

TABLE 11 : LOGIN FLOW VALIDATION: 37

CamScanner

CHAPTER 1

INTRODUCTION

CamScanner

1.1 Introduction:

In today's digitally-driven academic landscape, the need for efficient, scalable, and secure educational tools has never been more urgent. Traditional classroom assessment methods are rapidly evolving as educators seek innovative ways to engage students and evaluate their knowledge. This thesis presents "Quiz Runner Over a Restricted Network", a streamlined, web-based quiz management system designed to simplify the teaching and testing process—particularly in environments with controlled or limited internet access. Quiz Runner enables teachers to generate, manage, and publish quizzes seamlessly from educational materials such as PDFs, using advanced AI-powered tools. Leveraging the power of OpenAI's GPT-4 model, the system intelligently extracts meaningful content from uploaded documents and transforms it into well-structured multiple-choice questions (MCQs). Teachers can set the duration of each quiz, manage student registration, and monitor performance—all from a user-friendly dashboard.

Students, on the other hand, can register under specific teachers, await approval, and then gain access to quizzes in real-time. Upon completing quizzes, their results are instantly recorded and made available to teachers for review and performance tracking.

What sets Quiz Runner apart is its ability to function reliably within restricted or secured network environments, such as educational institutions with firewalls or bandwidth limitations. Built with Streamlit for the frontend, Firebase for real-time data handling, and powered by robust backend components including OpenAI APIs and PyMuPDF for PDF parsing, the system ensures speed, security, and scalability.

This thesis explores the design, development, and deployment of Quiz Runner, demonstrating how AI and modern web technologies can come together to revolutionize digital assessment—making it more accessible, automated, and intelligent.

1.2 Security:

In the development of any web-based application especially one dealing with sensitive educational data such as quiz content, student credentials, and performance records security is paramount. The Quiz Runner system is designed with multiple layers of security mechanisms to protect both user data and system integrity, particularly within restricted or institutionally controlled networks.

1.2.1 User Authentication and Access Control:

Quiz Runner implements secure authentication protocols using Firebase Authentication integrated with Google's Identity Toolkit. Each user, whether a teacher or a student, is required to create an account using a valid email and password. Roles are distinctly defined and enforced within the system to ensure that teachers and students can access only their respective dashboards and functionalities. Unauthorized access is proactively blocked through role-based access checks at every sensitive entry point.

CamScanner

1.2.2 Encrypted Data Handling:

All user credentials are transmitted over HTTPS and securely managed by Firebase, which adheres to Google's industry-grade security standards. Internally, sensitive data such as passwords are hashed using bcrypt, a secure one-way hashing algorithm, to prevent plaintext storage and exposure.

1.2.3 Firestore Security Rules:

Firebase Firestore, the database backbone of Quiz Runner, is protected using custom security rules. These rules strictly govern read/write operations based on the user's identity and role. For instance, only a teacher can access their own students' data and quiz results, while students can only view or submit quizzes assigned to them.

1.2.4 Restricted Network Compatibility:

Recognizing that many educational institutions operate within restricted network environments, Quiz Runner is optimized to perform reliably with minimal external dependencies. Key operations, such as MCQ generation via OpenAI APIs and quiz data transactions via Firebase, are streamlined to maintain functionality even under limited bandwidth or firewall constraints. This minimizes the application's attack surface and enhances its resilience in controlled settings.

1.3 Background of the Project:

In today's digital era, the transformation of traditional educational practices into modern, technology-driven solutions has become a critical component of academic institutions worldwide. The rapid adoption of e-learning tools, online assessments, and remote teaching platforms has significantly improved accessibility and efficiency. However, these advancements are often hindered in environments with restricted network access, such as government institutions, examination centers, or schools with strict firewall policies. These constraints limit the use of many conventional online platforms, posing challenges for conducting secure and scalable assessments.

The Quiz Runner project was conceptualized to bridge this technological gap. It is specifically designed to facilitate real-time quiz creation, distribution, and evaluation within constrained or restricted networks, without compromising on usability or security. The core idea emerged from the observed need for a lightweight, role-based quiz management system that operates efficiently in such environments, while still offering the modern conveniences of cloud-based tools.

This system empowers teachers to upload content (in PDF format), automatically generate multiple-choice questions using GPT-powered intelligence, and instantly publish quizzes to students. On the other side, students can seamlessly register, connect with teachers, attempt quizzes, and view their results, all through a simplified and intuitive interface.

CamScanner

By integrating OpenAI's GPT-4 for intelligent MCQ generation, Firebase for secure user and data management, and Streamlit for an interactive UI, Quiz Runner represents a balanced fusion of AI, security, and accessibility tailored for educational institutions with infrastructure limitations.

This project not only responds to an immediate academic need but also sets a foundation for future-ready, AI-enhanced learning tools that can operate within restrictive digital landscapes, making quality assessment tools universally accessible.

1.4 Scope of the Project:

The **Quiz Runner** project is designed with a focused yet flexible scope to support educational assessments in **environments with limited or restricted internet access**. Its primary aim is to provide a **secure, user-friendly, and AI-powered quiz platform** that operates effectively in constrained network conditions while maintaining modern educational standards.

Key Inclusions in Scope:

- **Teacher-Focused Features**:
  - Secure registration and authentication for teachers.
  - PDF-based content upload with automated text extraction.
  - AI-generated multiple-choice questions (MCQs) using OpenAI's GPT-4.
  - Real-time quiz publishing with customizable duration.
  - Student management including viewing registered learners and handling registration requests.
  - View and manage quiz results.
- **Student-Focused Features**:
  - Student registration and login with secure access.
  - Registration requests to connect with specific teachers.
  - Live access to published quizzes with a countdown-based interface.
  - Automatic result calculation and result history tracking.

CamScanner

1.5 Feasibility of the Project:

The development and deployment of the **"Quiz Runner Over a Restricted Network"** system have been assessed from various feasibility perspectives. This section outlines the economic, technical, and operational feasibility to justify the viability of the proposed solution.

1.5.1 Economic Feasibility:

The project is economically viable, with minimal cost requirements compared to traditional large-scale assessment systems. By utilizing open-source tools, cloud infrastructure, and existing network capabilities within educational institutions, the financial investment is substantially reduced.

Key Considerations:

- **No Licensing Costs**: Technologies such as Streamlit, Firebase (on free-tier), and PyMuPDF are open-source or offer generous free plans.
- **Cloud Backend (Firebase)**: Allows data storage and user management at low to no cost for academic use cases.
- **Quiz Generation (OpenAI API)**: Managed usage and optimization ensure predictable and affordable token-based billing.
- **Maintenance**: Streamlined architecture reduces the need for dedicated maintenance personnel.

1.5.2 Technical Feasibility:

The project has been successfully implemented using modern, lightweight technologies compatible with low to moderate computing environments.

Hardware Requirements:

- Standard desktop or laptop systems for teachers and administrators.
- Basic Android tablets or PCs for students.
- Local server (optional) for deployment in completely offline/restricted environments.

Software Stack:

- **Frontend**: Streamlit (Python)
- **Backend**: Firebase Firestore (NoSQL DB)
- **Quiz Generation**: OpenAI's GPT-4 via API

CamScanner

1.5 Feasibility of the Project:

The development and deployment of the **"Quiz Runner Over a Restricted Network"** system have been assessed from various feasibility perspectives. This section outlines the economic, technical, and operational feasibility to justify the viability of the proposed solution.

1.5.1 Economic Feasibility:

The project is economically viable, with minimal cost requirements compared to traditional large-scale assessment systems. By utilizing open-source tools, cloud infrastructure, and existing network capabilities within educational institutions, the financial investment is substantially reduced.

Key Considerations:

- **No Licensing Costs**: Technologies such as Streamlit, Firebase (on free-tier), and PyMuPDF are open-source or offer generous free plans.
- **Cloud Backend (Firebase)**: Allows data storage and user management at low to no cost for academic use cases.
- **Quiz Generation (OpenAI API)**: Managed usage and optimization ensure predictable and affordable token-based billing.
- **Maintenance**: Streamlined architecture reduces the need for dedicated maintenance personnel.

1.5.2 Technical Feasibility:

The project has been successfully implemented using modern, lightweight technologies compatible with low to moderate computing environments.

Hardware Requirements:

- Standard desktop or laptop systems for teachers and administrators.
- Basic Android tablets or PCs for students.
- Local server (optional) for deployment in completely offline/restricted environments.

Software Stack:

- **Frontend**: Streamlit (Python)
- **Backend**: Firebase Firestore (NoSQL DB)
- **Quiz Generation**: OpenAI's GPT-4 via API

CamScanner

- **PDF Processing**: PyMuPDF
- **Authentication**: Firebase Authentication with email/password and role management

Expertise Required:

- Intermediate Python development skills
- Basic familiarity with Streamlit and Firebase
- Understanding of REST APIs and database management

Given the availability of skilled developers and the modular nature of the stack, the technical requirements are manageable and do not pose any major barriers.

1.6 Introduction of the Tools Used in the Project:

The development of the **"Quiz Runner Over a Restricted Network"** system was achieved through the integration of various powerful development tools, libraries, and platforms. Each tool was chosen for its unique capabilities, ease of use, and compatibility with modern software development practices. This section outlines and discusses each tool individually.

1.6.1 Streamlit:

Purpose: Web App Framework

Description:
Streamlit is an open-source Python library that simplifies the creation of interactive web applications for data science and machine learning. In this project, it serves as the frontend interface for both teachers and students. It enables rapid development and deployment of web pages for quiz creation, login/authentication, student registration, and result display, all without needing traditional HTML/CSS.

Key Features Utilized:

- Tabbed UI for dashboard navigation
- File uploader for PDF input
- User-friendly input forms and feedback alerts

1.6.2 Firebase (Firestore & Authentication):

Purpose: Backend as a Service (BaaS)

CamScanner

Description:
Firebase provides real-time database capabilities and robust user authentication. Two core services are used:

a. Firebase Firestore

A NoSQL cloud database used to store:

- User profiles (teachers and students)
- Quiz metadata and questions
- Quiz results and student submissions
- Teacher-student registration requests

b. Firebase Authentication

Used to manage secure login and signup with email/password credentials and role-based access (Teacher or Student).

1.6.3 Visual Studio Code (VS Code):

Purpose: Integrated Development Environment (IDE)

Description:
Visual Studio Code is the primary development environment used in this project. It offers a powerful and user-friendly interface with features such as:

- IntelliSense (code auto-completion and syntax suggestions)
- Git integration for version control
- Terminal access for running Streamlit apps
- Extensions for Python, Firebase, and AI tools
- Real-time error detection and debugging support

VS Code significantly enhanced development efficiency and code organization throughout the project lifecycle.

1.6.4 OpenAI GPT-4 API:

Purpose: AI-based MCQ Generation

CamScanner

Description:
OpenAI's GPT-4 model is utilized to generate high-quality multiple-choice questions from academic PDFs. The system extracts relevant text from the uploaded file and sends it to

GPT-4, which returns structured questions, correct answers, and plausible options.

Benefits:

- Contextually relevant question generation
- Eliminates manual quiz creation effort
- Ensures uniqueness and diversity in quizzes

1.6.5 PyMuPDF (fitz):

Purpose: PDF Text Extraction

Description:
PyMuPDF is a powerful library used for reading and extracting text from PDF documents. It is responsible for parsing the content of the uploaded study materials (PDFs) and providing clean, readable text for the AI quiz generator.

1.6.6 Python:

Purpose: Core Programming Language

Description:
Python serves as the primary programming language for the entire system. Its versatility, vast ecosystem, and support for machine learning, web development, and cloud services make it ideal for this application.

Key Uses:

- Backend logic and database integration
- Quiz generation using OpenAI API
- Real-time user interface with Streamlit

1.6.7 Requests Library:

Purpose: API Communication

Description:
The requests library in Python is used to interact with Firebase's REST API endpoints, especially for

CamScanner

operations like user signup, login, and password reset. It provides a reliable and simple interface for sending HTTP requests.

1.6.8 Bcrypt:

Purpose: Password Hashing (Optional)

Description:
bcrypt can be used to hash passwords for enhanced local security, though Firebase Authentication manages secure hashing by default. It may serve in future expansions where local authentication is required.

1.6.9 Python-dotenv:

Purpose: Environment Configuration

Description:
This tool is used to securely load sensitive information like API keys (e.g., OpenAI, Firebase) from .env files, improving security and environment management.

1.7 Report of Introduction:

This report presents the design, development, and evaluation of the project titled **"Quiz Runner Over a Restricted Network."** The initial chapters begin with a comprehensive introduction to the project's objectives, followed by a discussion of the system's security architecture and its relevance in safeguarding academic integrity. The background chapter outlines the motivation and technological context that led to the development of this solution. The scope chapter highlights the intended functionality and the user groups involved. A detailed overview of the application domain, particularly focusing on its relevance within the educational institutions of Pakistan, is also provided. The feasibility chapter assesses the technical, economic, and operational viability of the system, while the tools section describes the programming environments, APIs, and platforms employed. The system design chapter covers the architectural flow, database structure, and interface layouts. Implementation details are discussed next, showing how core functionalities were realized. Finally, the report concludes with testing results, limitations, and recommendations for future enhancements, ensuring a complete and structured overview of the project lifecycle.

CamScanner

CHAPTER 02

REQUIREMENTS SPECIFICATION

CamScanner

2.1 Introduction:

Requirement Specification is a critical phase in the software development lifecycle where the needs of the system's stakeholders are analyzed and documented in a clear and structured manner. For the **"Quiz Runner Over a Restricted Network"** project, this phase establishes the functional and non-functional requirements that guide the development of a secure, efficient, and user-friendly quiz management platform tailored for educational environments. This chapter outlines the specific capabilities that the system must provide for teachers and students, ensuring that all components—from quiz generation and publication to student registration, quiz attempts, and result management—are thoroughly defined. Moreover, it specifies performance expectations, user access restrictions, usability considerations, and security standards necessary for operating within constrained or controlled networks such as those found in schools, colleges, or training institutions. By formally documenting these requirements, this chapter lays a solid foundation for the design and implementation phases that follow.

2.2 Existing System:

In many educational institutions, especially within Pakistan, traditional methods of conducting quizzes and assessments are still prevalent. These existing systems often rely on paper-based tests or basic online platforms that lack integration, automation, and security. While some institutions use Learning Management Systems (LMS) or generic quiz apps, these tools generally require stable internet connectivity and lack customized features tailored for local use, especially in restricted or low-connectivity environments.

2.3 Limitations of the Existing System:

The current systems used for conducting quizzes and assessments whether manual or generic online platforms come with a range of limitations that hinder efficiency, scalability, and fairness in the evaluation process. These limitations are particularly evident in environments with restricted internet access or limited technological infrastructure, such as many educational institutions in developing regions. The major limitations are as follows:

2.3.1 High Dependence on Internet Connectivity:

Most existing online quiz systems require stable and uninterrupted internet access. This becomes a major challenge in rural or semi-urban areas where internet access is limited, unstable, or costly.

2.3.2 Lack of Automation:

Manual quiz creation and grading is time-consuming and prone to human error. Even digital platforms often do not offer automated question generation or real-time evaluation features, increasing the workload on educators.

CamScanner

2.3.3 Security Vulnerabilities:

Generic platforms often lack robust authentication mechanisms, making them vulnerable to unauthorized access, impersonation, and cheating during online assessments.

2.3.4 Inability to Generate Smart MCQs:

Manual question generation from study material or PDFs not only requires significant time but also lacks consistency. Most systems do not utilize artificial intelligence to assist in creating meaningful and varied multiple-choice questions.

2.3.5 Lack of Quiz Timing and Duration Control:

Many tools fail to offer precise control over quiz availability and timing. Either quizzes are available indefinitely or lack real-time duration management, reducing fairness and structure in assessments.

2.4 Proposed System:

To overcome the challenges and limitations posed by existing quiz and assessment systems, this project proposes a robust, secure, and intelligent solution titled "Quiz Runner Over a Restricted Network." The system is designed to operate efficiently in environments with limited internet access, such as schools, colleges, and examination centers located in remote areas. It facilitates seamless quiz creation, publishing, and evaluation through an AI-powered, role-based web application developed using modern technologies.

The proposed system introduces a dual-role architecture comprising Teachers and Students, each with dedicated functionalities. Teachers can upload content in PDF format and utilize integrated AI (ChatGPT-4 via OpenAI API) to automatically generate high-quality multiple-choice questions (MCQs). These quizzes can be published with controlled timing (duration-based rather than calendar-bound scheduling), and results are evaluated automatically upon submission. Students, on the other hand, register under a specific teacher through a request-based approval system.

2.5 Functional Requirements:

Functional requirements define the specific behavior, operations, and activities the proposed system must be able to perform to meet user needs. For the "Quiz Runner Over a Restricted Network" project, the system is divided into two major user roles: Teacher and Student. Each role includes a distinct set of functions and permissions to ensure proper flow and management of quizzes within the application.

CamScanner

2.5.1 User Authentication and Account Management:

User Registration

- Users must be able to register as either a **Teacher** or a **Student**.
- During registration, the user must provide an email address, name, password, and role.
- The system stores user credentials and role-based metadata in Firebase.

User Login

- All users must be able to log in using their registered email and password.
- Role-based access control is enforced: users are redirected to their respective dashboards upon login.

Password Recovery

- A password reset option allows users to receive a reset link via email.
- Firebase handles the secure delivery and management of password resets.

2.5.2 Teacher Functionalities:

Upload Educational Content (PDF)

- Teachers can upload a PDF file containing content from which quiz questions are generated.
- The system uses PyMuPDF to extract text from the uploaded PDF.

Generate MCQs

- After uploading, teachers specify the number of questions to generate.
- The system uses OpenAI GPT-4 to analyze the text and generate multiple-choice questions (MCQs).
- Each question includes:
    - A clear question
    - A correct answer

CamScanner

- Three distractor options

Publish Quiz

- Teachers can publish a quiz by specifying:
  - Quiz title
  - Duration in minutes
- The quiz becomes instantly available to students upon publishing.
- The system records the quiz's metadata and availability window in Firestore.

Manage Student Registrations

- Teachers can view and approve or reject pending student registration requests.
- Once approved, the student becomes associated with the teacher's profile.

View Registered Students

- A list of all students under a teacher is displayed, including names and email addresses.

View Student Results

- Teachers can view individual student scores for each quiz.
- Submission timestamps and scores are shown for accountability.

Delete Quiz

- A teacher can delete a quiz only if it has not been attempted by any student.
- Once a quiz has responses, deletion is disables to preserve data integrity.

2.5.3 Student Functionalities:

Send Registration Request

- Students select a teacher and send a request for approval.
- Teachers must approve this request before students can access their quizzes

View Available Quizzes

CamScanner

- Students can view a list of quizzes published by their approved teacher.
- Only quizzes currently within their valid duration are shown.

Attempt Quiz

- Students can attempt each quiz **only once**.
- The quiz must be completed within the allotted duration.
- Questions are presented one by one or all at once depending on UI design.

Automatic Evaluation

- The system evaluates the quiz automatically by comparing student answers with the correct ones.
- A final score is calculated and stored in Firestore.

View Past Results

- Students can view a history of attempted quizzes along with:
    - Quiz title
    - Score achieved
    - Submission date

Delete Results

- Students have the option to delete their past results from their profile if desired

2.5.4 Administrative Functionalities:

User Data Management

- Admins can view, add, or remove users through Firebase Console.

Quiz and Submission Tracking

- All quizzes and submissions are stored in Firestore for transparency and auditability.

CamScanner

2.5.5 Use case Diagram:

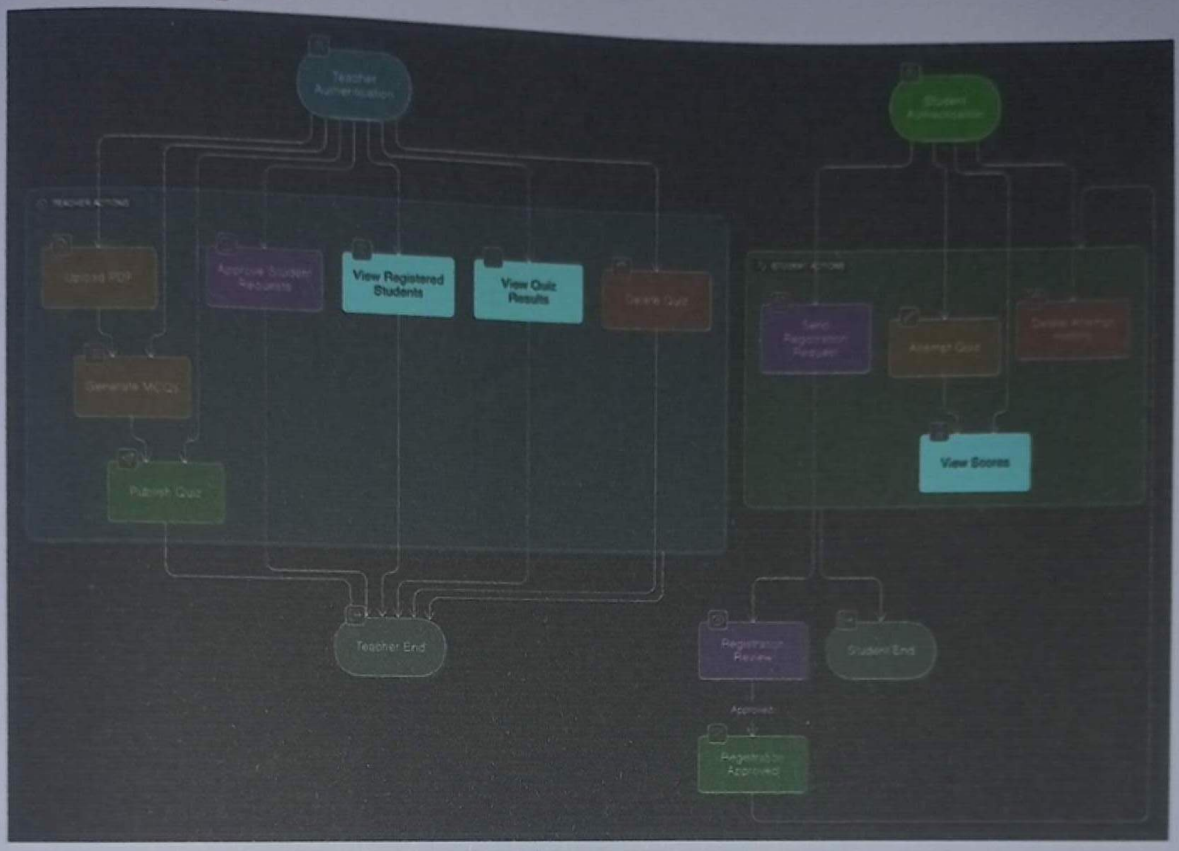


Figure 2.1 Use Case Diagram

CamScanner

2.5.6 Data Flow Diagram:

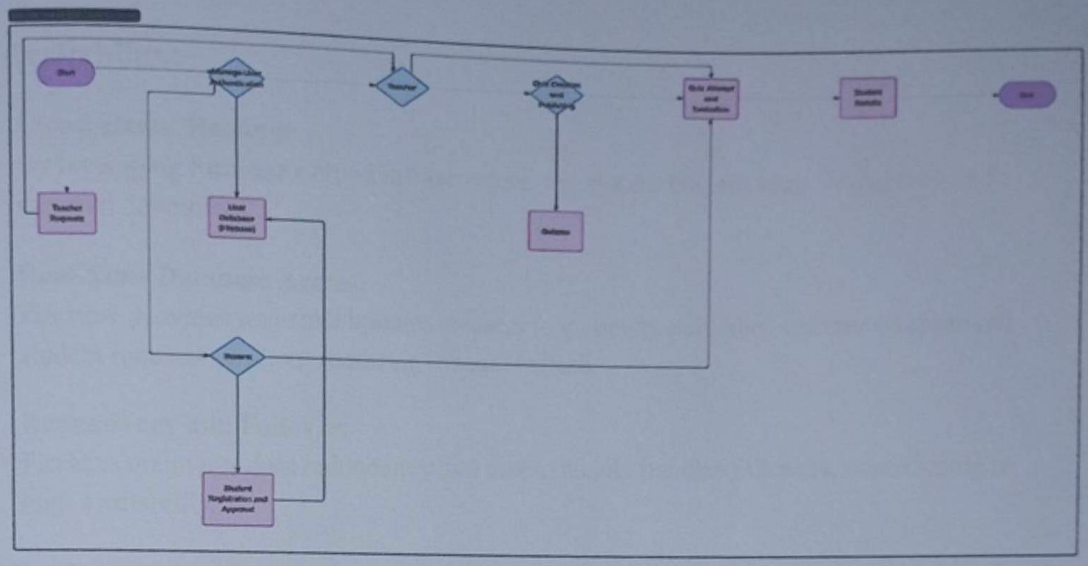


Figure 2.2 : Data Flow Diagram

2.6 Non-Functional Requirements:

Non-functional requirements define the system's quality attributes and constraints that ensure the system performs effectively, efficiently, and securely. These requirements are crucial for the overall success, user experience, and long-term sustainability of the Quiz Runner Over a Restricted Network system.

2.6.1 Security:

- **Authentication and Authorization:**
  The system uses Firebase Authentication to enforce secure, role-based access. Only authenticated users can access teacher or student dashboards.

- **Data Privacy:**
  Sensitive user information, such as passwords, is securely handled using Firebase's encryption and hashing mechanisms.

- **Restricted Access:**
  Students can only access quizzes published by the teacher they are registered under, ensuring exam confidentiality.

- **One-Time Attempt Restriction:**
  Each quiz can be attempted only once per student, and responses are locked after submission.

CamScanner

2.6.2 Availability:

- **Cloud-Based Hosting:**
  By leveraging Firebase's cloud infrastructure, the system ensures high availability and minimal downtime.

- **Real-Time Database Access:**
  Firestore provides real-time updates to users (e.g., newly published quizzes or approved student requests) without requiring manual refresh.

- **Redundancy and Failover:**
  Firebase maintains data redundancy and automatically handles failovers, contributing to high availability.

2.6.3 Performance:

- **Efficient Data Handling:**
  Firestore enables fast read/write operations, even under concurrent user sessions, ensuring responsive user experiences.

- **MCQ Generation Optimization:**
  The use of OpenAI's GPT-4 API ensures high-quality question generation with minimal processing time.

- **Minimal Latency in Evaluation:**
  Quiz results are evaluated and stored instantly upon submission, reducing user wait time.

2.6.4 Scalability:

- **User Growth Support:**
  Firebase and OpenAI APIs are capable of handling growth in user base, quizzes, and concurrent requests without major re-engineering.

- **Quiz Volume Expansion:**
  Teachers can create and manage multiple quizzes without performance degradation.

- **Multi-School/Institution Integration:**
  The architecture supports expansion to other institutions or departments with minimal configuration.

CamScanner

2.6.5 Maintainability:

- **Modular Codebase:**
  The system is built with modular Python scripts (auth.py, quiz_db.py, teacher_dashboard.py, etc.), enabling easy maintenance and upgrades.
- **Cloud-Based Configuration:**
  Changes to data structure, user roles, or quiz logic can be done centrally using Firebase's console, reducing deployment overhead.

2.6.6 Usability:

- **Simple and Intuitive UI:**
  The Streamlit-based frontend is user-friendly and accessible to both tech-savvy and non-technical users.
- **Mobile Accessibility:**
  The system can be accessed on mobile browsers, enabling students and teachers to interact with the system on the go.

2.6.7 Compatibility:

- **Cross-Platform Support:**
  As a web-based application, the system is compatible with all modern browsers and operating systems (Windows, macOS, Linux).
- **API Integration Compatibility:**
  The use of standard REST APIs (OpenAI, Firebase) ensures ease of integration with other third-party systems or future features.

CamScanner

CHAPTER 3

DESIGN OF THE PROPOSED SYSTEM

CamScanner

3.1 System Architecture:

The system architecture of the Quiz Runner Over a Restricted Network project is designed to support secure, efficient, and user-friendly online assessments in environments with limited internet access. The architecture ensures clear separation between frontend, backend, and database layers while maintaining centralized control and secure communication between different user roles teachers and students.

3.1.1 Components:

A. Frontend Interface (Streamlit)

A lightweight, web-based UI built using Streamlit that handles interactions with both teachers and students:

- **Teacher Dashboard**: Allows uploading PDFs, generating quizzes using AI, viewing students, and reviewing results.
- **Student Dashboard**: Enables quiz participation, registration under teachers, and result tracking.
- **Authentication Interface**: Email/password login/signup using Firebase Authentication.

B. Business Logic Layer

Handles the core functionalities of the application:

- **MCQ Generation**: Uses **OpenAI's GPT-4 API** to convert extracted PDF content into quality multiple-choice questions.
- **PDF Processing**: Leverages **PyMuPDF (fitz)** to extract readable and structured text from uploaded PDF files.
- **Session Management**: Maintains user sessions and access based on role (Teacher/Student).
- **Time Management**: Automatically timestamps quizzes and restricts access after the duration ends.

C. Quiz Management Module

Used primarily by **teachers** to:

CamScanner

- Create and Publish Quizes.
- Manage registered students and pending requests.
- View performance reports and quiz analytics.

D. Student Interaction Module

Used by **students** to:

- Register under a specific teacher.
- Attempt quizzes only during the permitted time window.
- View their own performance and submission history.

E. Firebase Backend

A fully managed, scalable backend that provides:

- **Firestore Database**:
  - Stores structured data such as quizzes, results, and user details.
  - Real-time updates enable efficient student-teacher interaction.
- **Firebase Authentication**:
  - Manages secure login/signup.
  - Supports role-based access control (Teacher/Student)

CamScanner

3.1.2 Architecture Diagram:

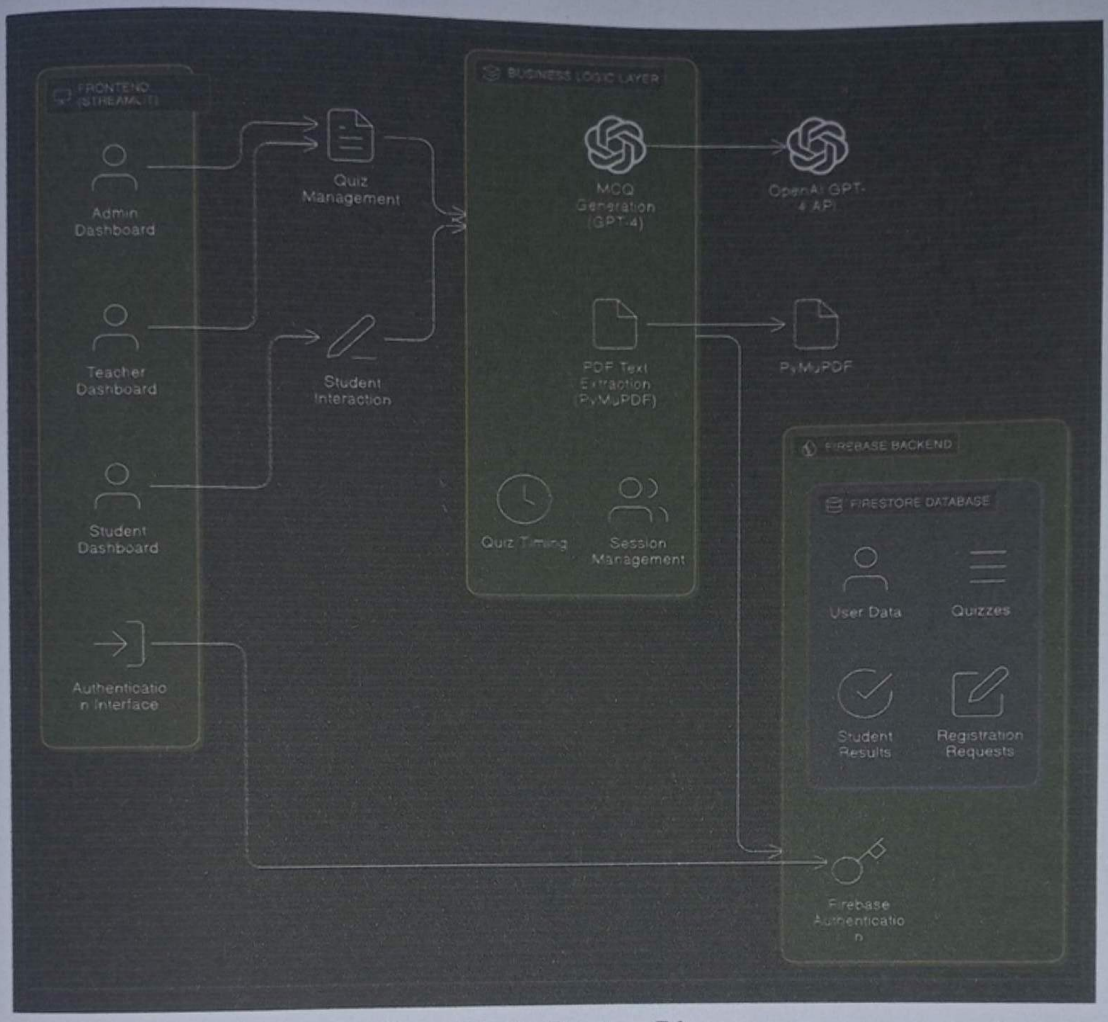


Figure 3.1 : Architecture Diagram

3.2 High-Level Design:

The logical architecture of the system defines how the core components interact logically separating functionalities into conceptual layers rather than focusing on physical deployment. It ensures clear responsibility distribution between components like the interface, control logic, and data management systems.

3.2.1 Presentation Layer:

Technology: Streamlit

Purpose: Acts as the user interface for students and teachers to interact with the system.

CamScanner

Components:

- Student Dashboard: View quizzes, submit answers, view results.
- Teacher Dashboard: Upload PDFs, generate and publish quizzes, manage student requests.

3.2.2 Application Logic Layer:

Technology: Python, OpenAI GPT-4 API, ThreadPoolExecutor, Streamlit session state

Purpose: Implements the core logic and workflows of the app.

Components:

- **MCQ Generator**: Generates quiz questions from uploaded text.
- **PDF Processor**: Extracts structured content from PDFs.
- **Access Control**: Handles role-based login sessions.
- **Quiz Timer**: Manages quiz start time and auto-calculated duration.
- **Result Engine**: Stores and evaluates quiz submissions.

3.2.3 Data Layer:

Technology: Firebase (Authentication + Firestore)

Purpose: Stores persistent data and manages user credentials securely.

Components:

- **Firestore**: NoSQL database for storing structured user, quiz, and result data.
- **Authentication**: Verifies user identity and ensures role-specific access.

3.2.4 Key Features of Logical Architecture

- **Scalable**: Easily extendable with new roles or features.
- **Separation of Concerns**: Each layer has a distinct responsibility.
- **Secure**: Firebase ensures secure access and role validation.

CamScanner

- **Cloud-Based**: Persistent data is stored securely and in real-time.

3.2.5 Logical Architecture Diagram:

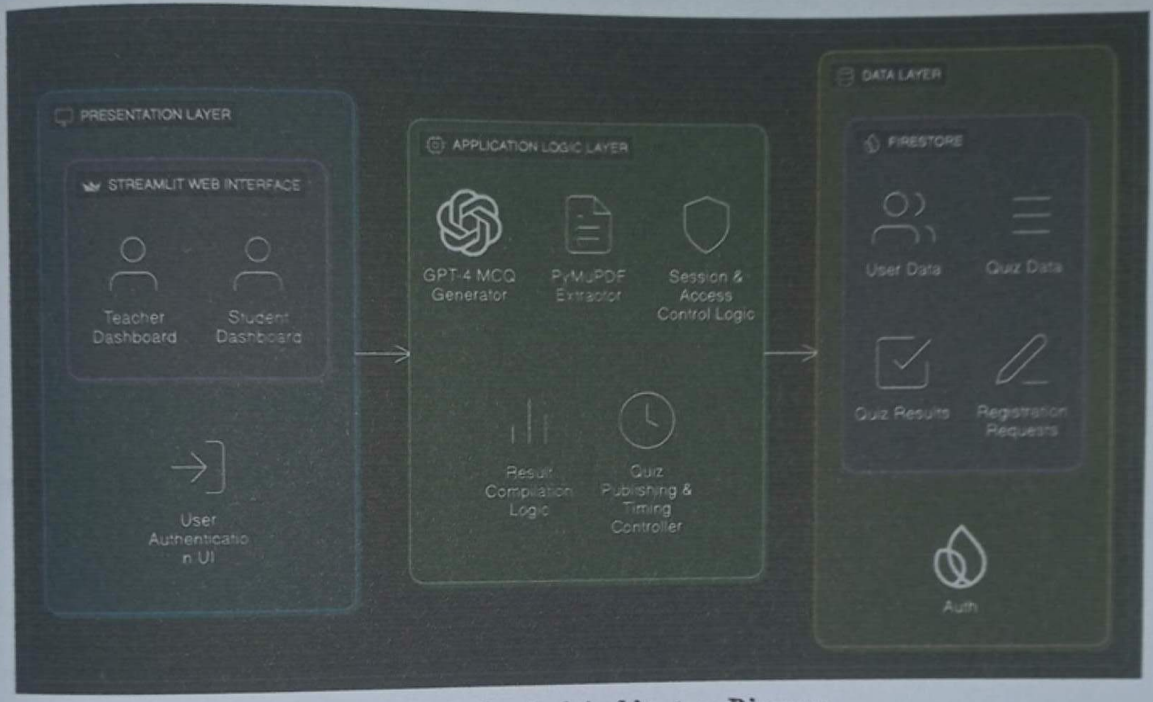


Figure 3.2 : Logical Architecture Diagram

3.3 Low Level Design:

The Low Level Design (LLD) focuses on the internal structure and implementation details of individual modules and classes. It provides a blueprint for developers to follow when implementing the functionalities described in the high-level design. The LLD uses class diagrams to represent object-oriented components and their relationships.

CamScanner

3.3.1 Class Diagram:

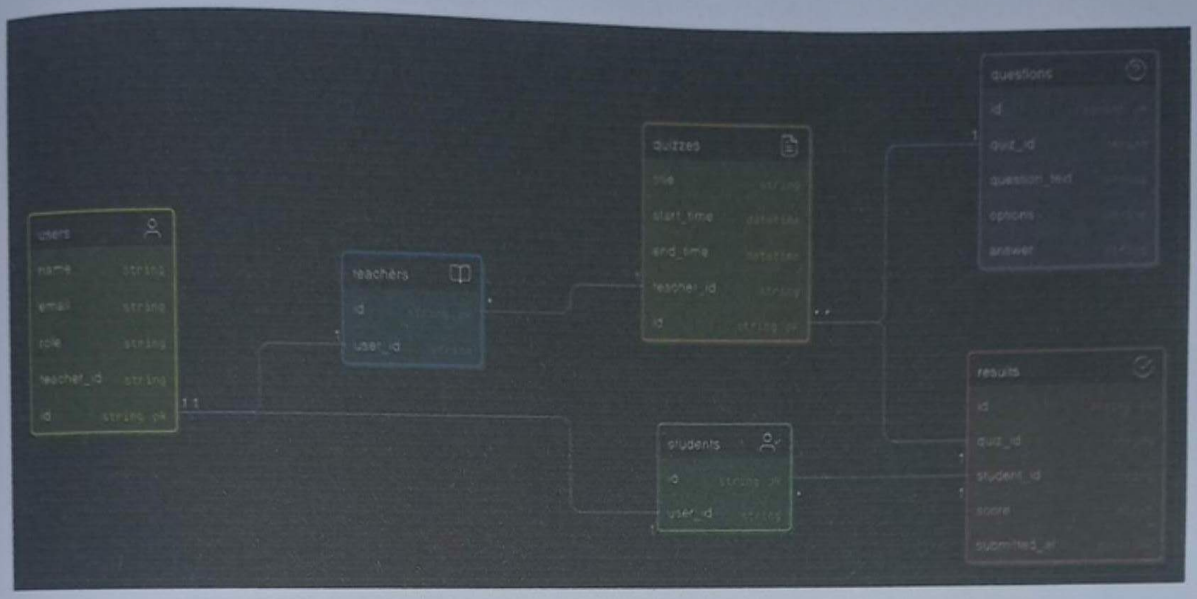


Figure 3.3 : Class Diagram

3.3.2 Class Descriptions:

User

- Represents the base user class with common attributes like uid, name, email, and role.
- Contains login, signup, and user info retrieval methods.

Teacher (Inherits User)

- Specialized class for teachers.
- Can create and publish quizzes, view registered students, and handle registration requests.

Student (Inherits User)

- Specialized class for students.
- Can send registration requests, attempt quizzes, and view past results.

Quiz

- Stores quiz metadata and questions.
- Contains methods for MCQ generation (via GPT-4), publishing, and deletion.

CamScanner

Question

- Represents a single MCQ.
- Contains question text, options, and correct answer.

Result

- Stores a student's submission result for a quiz.
- Includes the score and timestamp of submission.

3.4 Optimized Database Design:

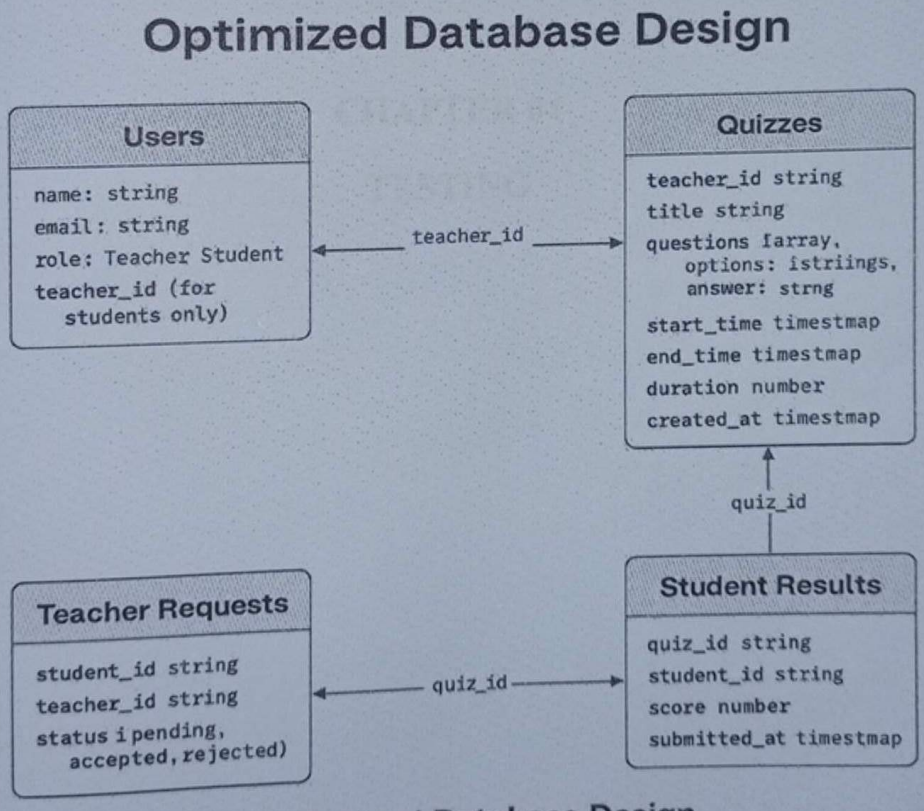


Figure 3.4 : Optimized Database Design

CamScanner

CHAPTER 04

TESTING

CamScanner

4.1 Test Case Scenarios:

4.1.1 Authentication Module:

Test Case 1:

- **User Signup - Successful Registration**

Scenario:
The user enters valid details (email, password, name, and role) and initiates the signup process.

Expected Result:
The system should successfully create the user account, store the data in Firebase, and return a confirmation message with the user ID.

Table 1: Authentication Module:

Test Case Title	**Successful Signup Test**
Use Case ID	01
Description	This test ensures that the user signup process works with valid input.
Cases	Validate Firebase Authentication API call. Verify data is saved in the Firestore database.
Test Results	Successfully executed.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

Test Case 2:

- **User Signup - Email Already Exists**

Scenario:
User tries to sign up using an email that already exists in the Firebase database.

Expected Result:
The system should return an appropriate error message indicating that the email already exists.

CamScanner

Table 2 : User Signup - Email Already Exists:

Test Case Title	Duplicate Email Signup Test
Use Case ID	02
Description	Ensures the system handles duplicate email signups.
Cases	Validate Firebase response for existing email. Return correct error to the user.
Test Results	Error handled correctly (EMAIL_EXISTS).
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

Test Case 3:

- **User Login – Successful**

Scenario:
User enters correct email and password to log in.

Expected Result:
System authenticates the user and returns a valid user ID.

Table 3 : User Login – Successful:

Test Case Title	Successful Login Test
Use Case ID	03
Description	Checks login functionality with valid credentials.
Cases	Confirm successful response from Firebase. Check if user ID is returned.
Test Results	Login successful.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

CamScanner

Test Case 4:

- **User Login – Incorrect Password**

Scenario:
User enters an incorrect password during login.

Expected Result:
System should return an error message indicating invalid credentials.

Table 4 : User Login – Incorrect Password:

Test Case Title	Failed Login Test
Use Case ID	04
Description	Validates system behavior on incorrect login attempts.
Cases	- Simulate invalid password input. - Handle Firebase error response correctly.
Test Results	Error message (INVALID_PASSWORD) returned.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

Test Case 5:

- **Password Reset - Valid Email**

Scenario:
User requests a password reset with a registered email.

Expected Result:
A password reset link is sent to the user's email.

CamScanner

Table 5 : Password Reset - Valid Email:

Test Case Title	Password Reset (Valid Email)
Use Case ID	05
Description	Tests reset functionality with a valid email.
Cases	- Confirm reset email is triggered. - Firebase response is successful.
Test Results	Email sent successfully.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

Test Case 6:

- **Password Reset - Invalid Email**

Scenario:
User tries to reset the password using an unregistered email.

Expected Result:
System returns an error message: EMAIL_NOT_FOUND.

Table 6 : Password Reset - Invalid Email:

Test Case Title	Password Reset (Invalid Email)
Use Case ID	06
Description	Ensures the system handles reset for unknown emails.
Cases	- Simulate non-existing email. - Validate proper error return from Firebase.
Test Results	Error handled correctly.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

CamScanner

Test Case 7:

- **Fetch User Info - User Exists**

Scenario:
System retrieves user data from Firestore for a valid user ID.

Expected Result:
Returns user details like name and email.

Table 7 : Fetch User Info - User Exists:

Test Case Title	Retrieve User Info (Exists)
Use Case ID	07
Description	Validates data retrieval from Firestore.
Cases	- Ensure Firestore returns correct user data.
Test Results	User data retrieved successfully.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

Test Case 8:

- **Fetch User Info - User Does Not Exist**

Scenario:
System tries to retrieve user data for a non-existent user ID.

Expected Result:
Returns None or appropriate null result.

CamScanner

Table 8 : Fetch User Info - User Does Not Exist:

Test Case Title	Retrieve User Info (Not Exists)
Use Case ID	08
Description	Confirms system behavior when user does not exist.
Cases	- Check Firestore query result. - Handle null or non-existing user.
Test Results	None returned as expected.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

4.1.2 MCQs Generation Module:

Test Case 1:

- **Generate MCQs from Input Text**

Scenario:
The system receives a paragraph of text and generates five MCQs using the question generation and question answering pipelines.

Expected Result:
The system should return a list of five MCQs. Each MCQ should contain:

- a question,
- a correct answer (retrieved from the context),
- and four options (1 correct + 3 distractors), with the correct answer included in the options

CamScanner

Table 9 : Generate MCQs from Input Text:

Test Case Title	MCQs Generation Test
Use Case ID	01
Description	Validates that MCQs are generated correctly using mocked NLP pipelines.
Cases	- Generate questions from the text. - Generate answers from context. - Ensure each MCQ has correct structure (question, answer, options). - Each options list should include 4 choices including the correct one.
Test Results	Successfully generated MCQs with all required fields and valid structure.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

4.1.3 PDF Text Extraction Module:

Test Case 1:

- **Extract Text from Uploaded PDF File**

Scenario:
A user uploads a PDF file. The system should read the PDF content and extract all text from each page using the fitz library (PyMuPDF).

Expected Result:
The system successfully reads the PDF file, extracts and concatenates text from all pages, and returns it as a single string.

CamScanner

Table 10 : Extract Text from Uploaded PDF File:

Test Case Title	PDF Text Extraction Test
Use Case ID	01
Description	Test to ensure the uploaded PDF is correctly read and parsed into text using PyMuPDF.
Cases	Open PDF using a file stream. - Iterate over pages and extract text. - Return combined text as a string. - Verify all components are triggered correctly.
Test Results	Successfully extracted and returned concatenated text from multiple pages.
Tested By	Khizer Dawood, Muhammad Uzair, Ramish Khan

4.1.4 User Authentication Module:

Test Case 1:

- **Login Flow Validation**

Scenario:

The system is launched, and a user enters their login credentials. The system should authenticate the user, retrieve their profile from Firestore, and show a personalized sidebar greeting with role information.

Expected Result:

The application successfully logs in the user, sets the session state, and displays "Welcome, Test Teacher" and "**Role:** Teacher" on the sidebar.

CamScanner

Table 11 : Login Flow Validation:

Test Case Title	Login Flow Test
Use Case ID	02
Description	Login interface test of the Teacher Dashboard application.
Cases	• It is also verified that sidebar renders correct name and role after login.
Test Results	Successfully
Tested by	Khizer Dawood, Muhammah Uzair, Ramish Khan

CamScanner

CHAPTER 05

RESULTS OF SCREENSHOTS

CamScanner

5.1 Login Screen:

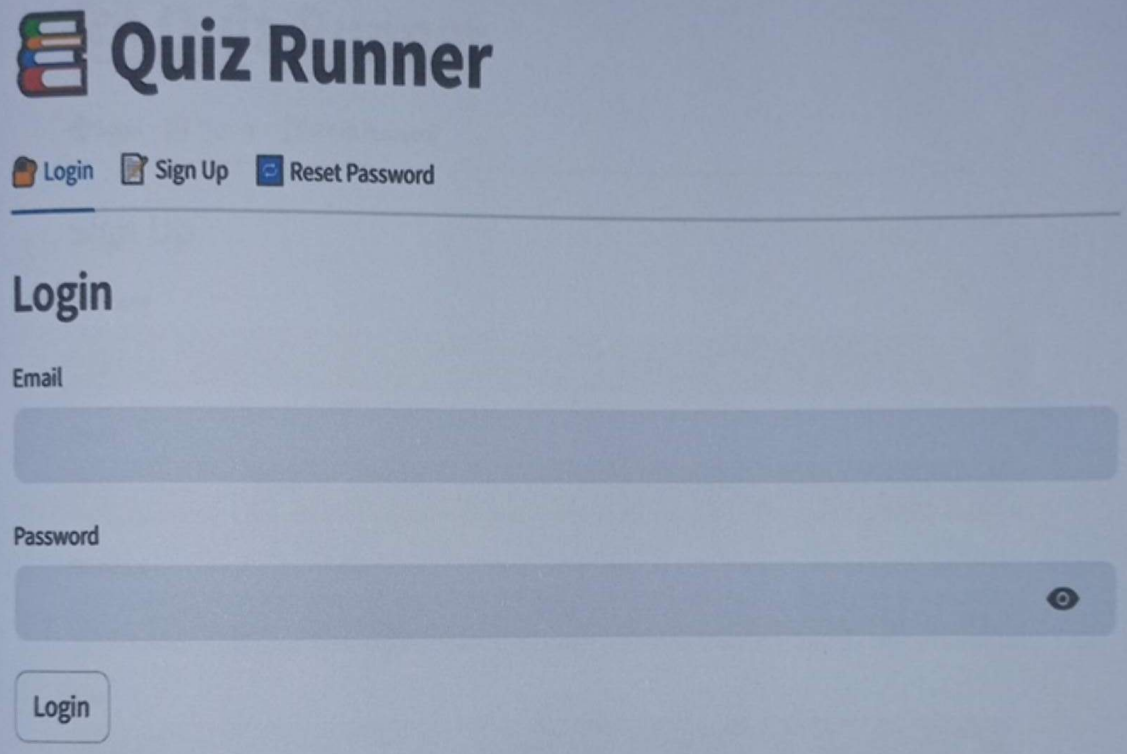


Figure 5.1 : Login Screen

CamScanner

5.2 Signup Screen:

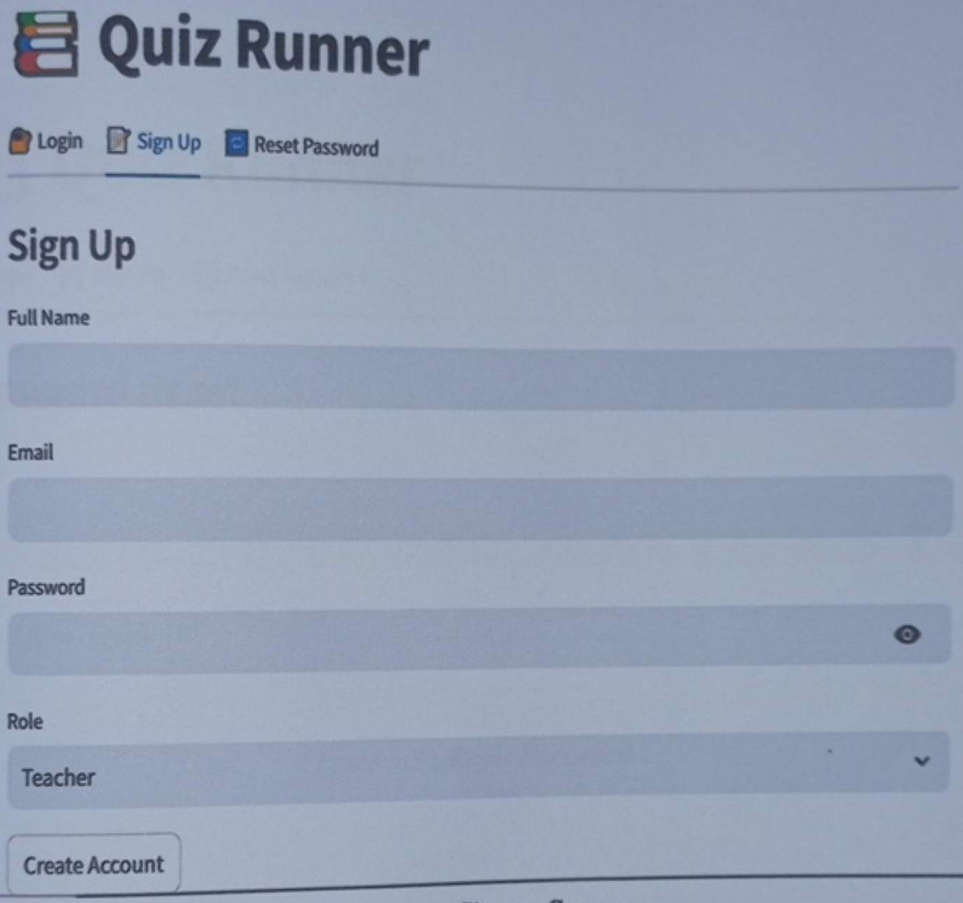


Figure 5.2 : Signup Screen

CamScanner

5.3 Reset Password:

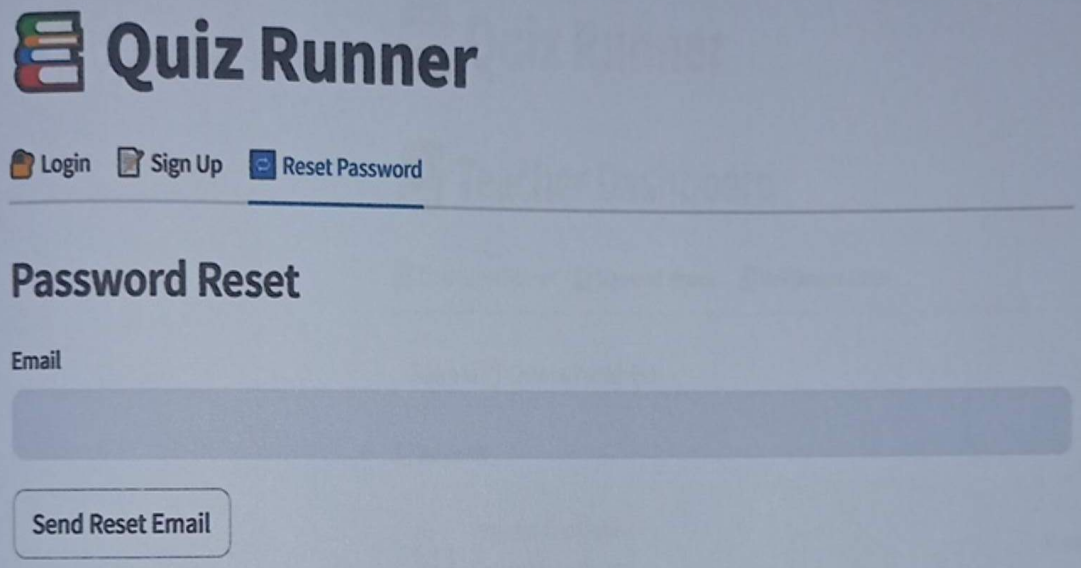


Figure 5.3 : Reset Password

CamScanner

5.4 Teacher Dashboard:

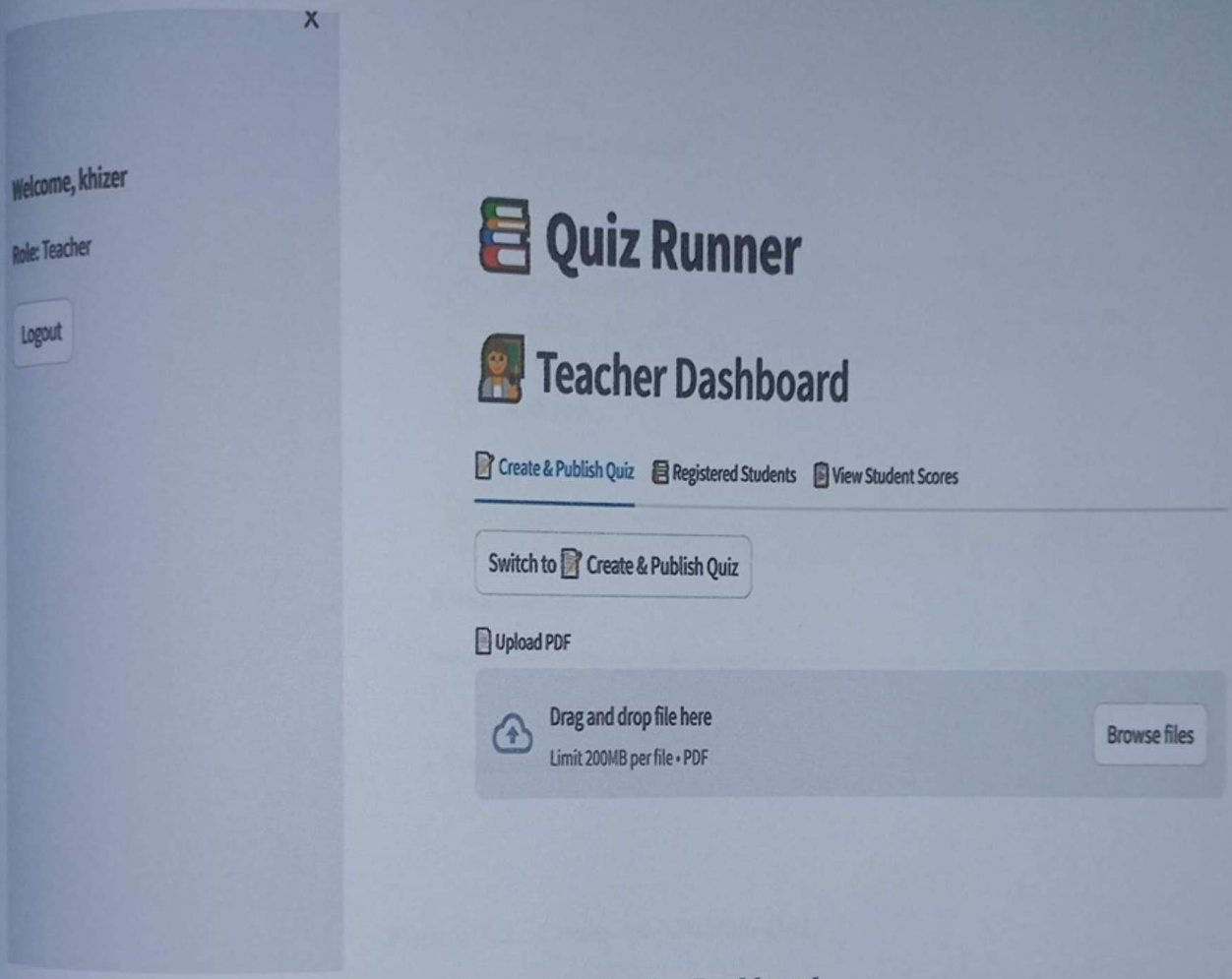


Figure 5.4 : Teacher Dashboard

CamScanner

5.5 Create And Publish Quiz:

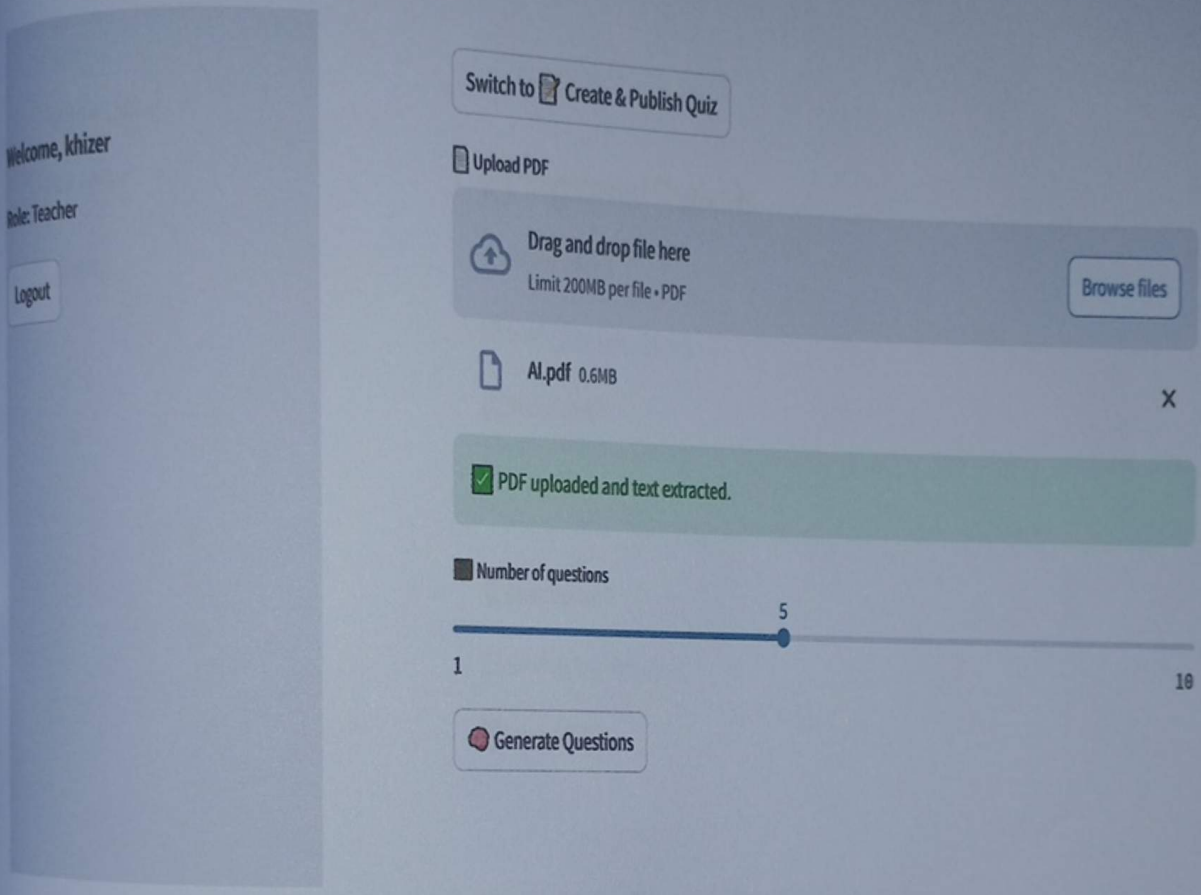


Figure 5.5 : Create and Publish Quiz

CamScanner

5.6 Register Students:

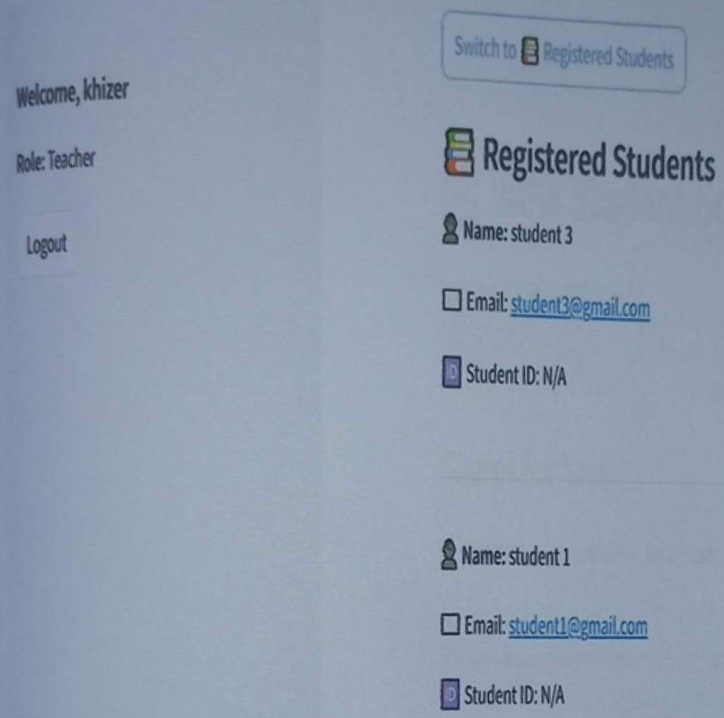


Figure 5.6 : Register Students

CamScanner

5.7 View Student Scores:

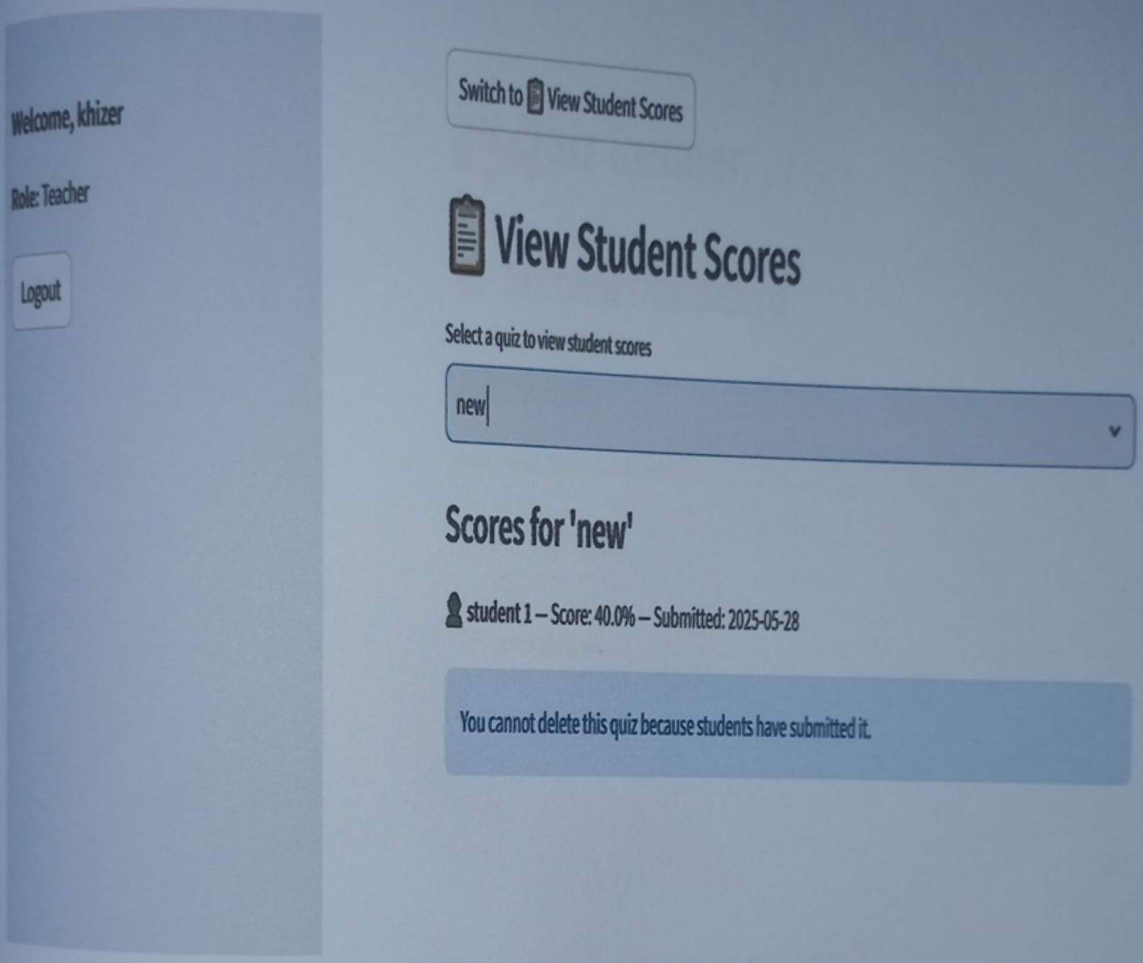


Figure 5.7 : View Student Scores

CamScanner

5.8 Student Dashboard:

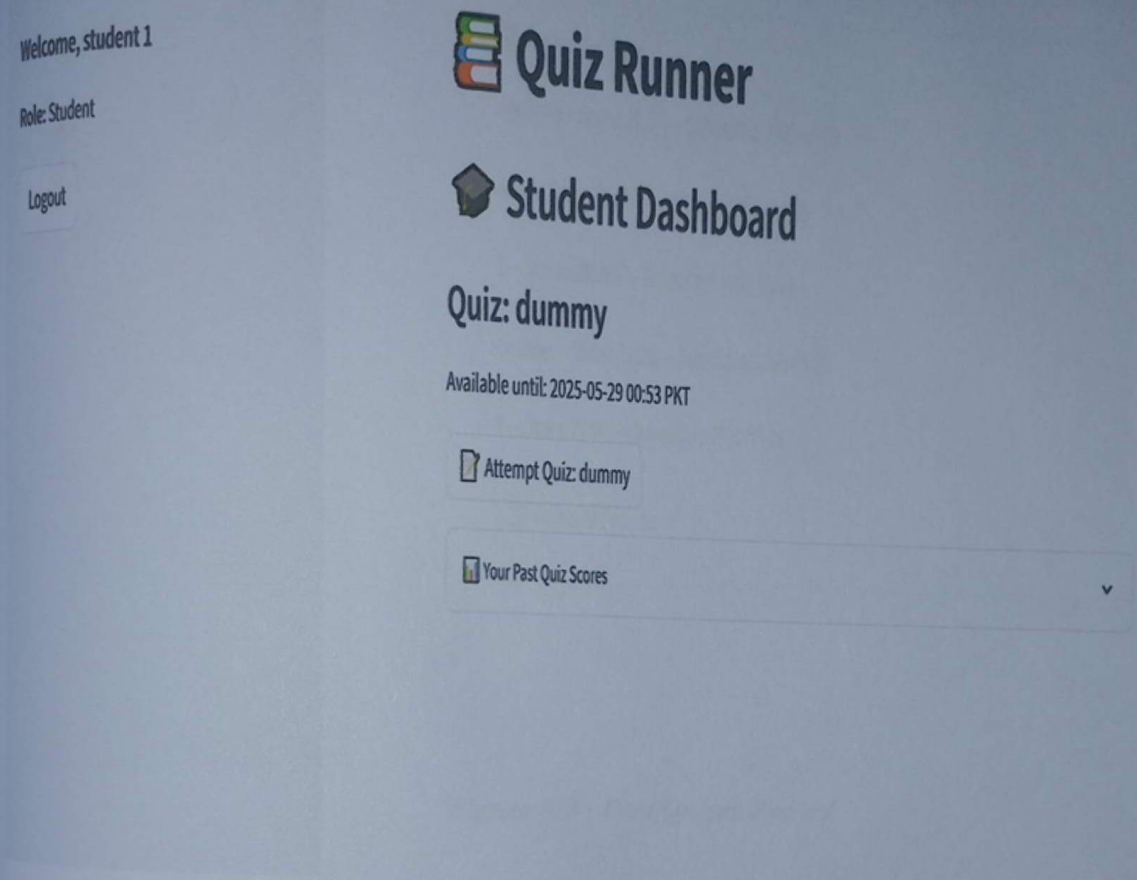


Figure 5.8 : Student Dashboard

CamScanner

5.9 Past Quizes Record:

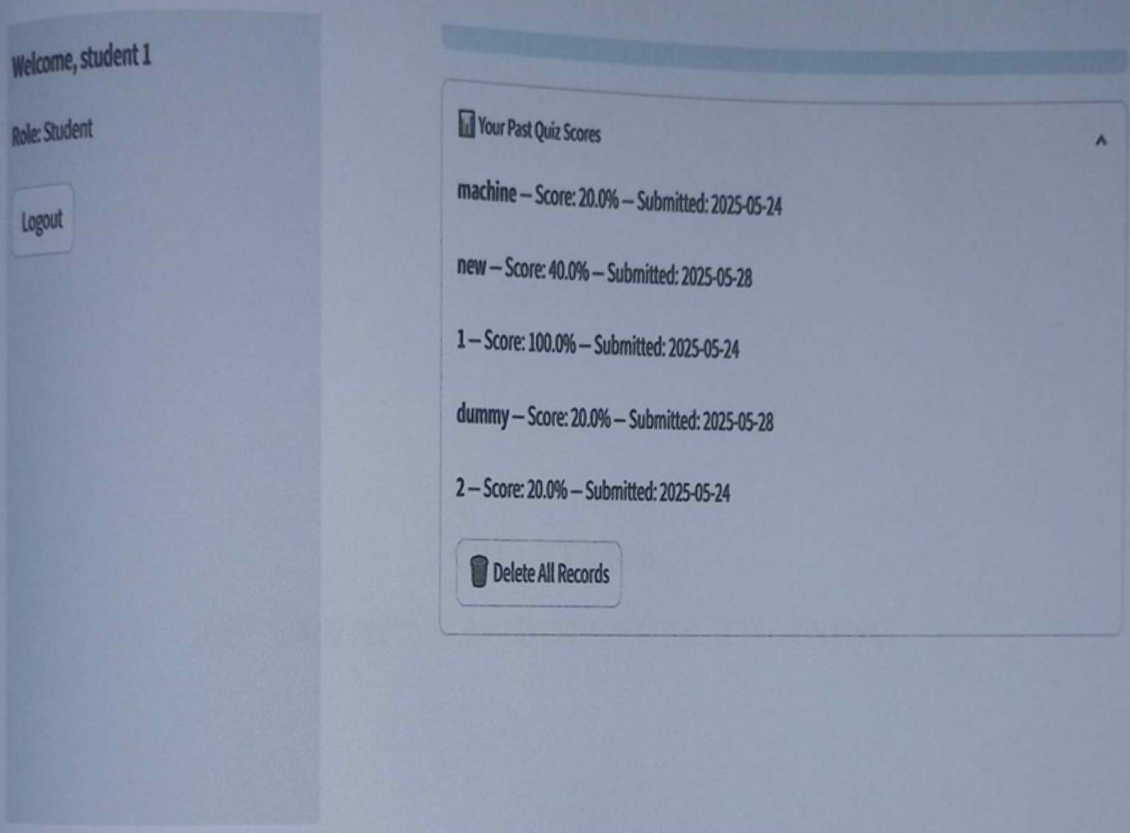


Figure 5.9 : Past Quizes Record

CamScanner

CHAPTER 6

REFERENCES AND MANUAL TRAINING

CamScanner

6.1 References:

[1] OpenAI, *GPT-3.5 Language Model*, [Online]. Available: https://openai.com

[2] Streamlit Inc., *Streamlit: The fastest way to build data apps*, [Online]. Available: https://streamlit.io

[3] Firebase, *Firebase Documentation*, Google, [Online]. Available: https://firebase.google.com/docs

[4] Google Cloud, *Firestore NoSQL Database*, [Online]. Available: https://cloud.google.com/firestore

[5] Python Software Foundation, *Python Programming Language*, Version 3.x, [Online]. Available: https://www.python.org

[6] Requests Library, *HTTP for Humans*, [Online]. Available: https://docs.python-requests.org

[7] PyMuPDF (fitz), *PDF Document Text Extraction*, [Online]. Available: https://pymupdf.readthedocs.io

[8] dotenv, *dotenv for Python*, [Online]. Available: https://pypi.org/project/python-dotenv/

[9] ThreadPoolExecutor, *Python concurrent.futures module*, [Online]. Available: https://docs.python.org/3/library/concurrent.futures.html

[10] Google Cloud Identity Platform, *Authentication via Firebase*, [Online]. Available: https://cloud.google.com/identity-platform

[11] GitHub, *Version Control and Repository Hosting*, [Online]. Available: https://github.com

[12] Markdown, *Lightweight Markup Language*, [Online]. Available: https://daringfireball.net/projects/markdown/

CamScanner

6.2 User Manual:

Version 1.0

1. Introduction

1.1 Purpose

Describes the aim of the Quiz Runner system and its intended users.

1.2 Scope

Defines the functionalities covered, such as quiz generation, restricted access, student-teacher interaction, and result viewing.

1.3 System Requirements

Outlines the hardware and software prerequisites:

- Web browser (Chrome/Firefox recommended)
- Internet access within a restricted network
- Python 3.x, Streamlit, Firebase backend

2. Getting Started

2.1 User Registration

Steps for both students and teachers to create an account.

2.2 User Login

Process of logging into the system securely using email and password.

2.3 Reset Password

Instructions to reset a forgotten password via email.

3. Teacher Dashboard

3.1 Generate and Publish Quiz

Upload PDF → Extract text → Generate MCQs → Publish with title and duration.

CamScanner

3.2 Registered Students

View and manage student registration requests. Accept or reject students.

3.3 View Student Scores

Access quiz results submitted by students, sorted by quiz and name.

4. Student Dashboard

4.1 Register Under the Teacher

Browse available teachers and send registration requests.

4.2 Attempt Published Quizzes

See and attempt quizzes published by accepted teachers within the allowed time.

4.3 View Scores

Access detailed results of attempted quizzes including correct answers.

4.4 Delete Previous Scores

Option to delete past attempts or reset quiz history.

CamScanner